Civic Monitor – Frontend Integration Guide

This document explains **ALL backend APIs**, **how to use them**, **in what order**, and **how UI components should interact with them**.

Backend is **cookie-based auth**, so frontend **must send cookies** (withCredentials: true).

🔐 AUTHENTICATION FLOW (MANDATORY FIRST)

🔹 1. Signup (Citizen)

API

POST /auth/signup

Payload

```
{
  "fullName": "John Doe",
  "email": "john@gmail.com",
  "password": "password123",
  "cityId": "<CITY_ID>",
  "zoneId": "<ZONE_ID>",
  "localityId": "<LOCALITY_ID>"
}
```

Frontend UI

- Signup form
- Dropdowns for City → Zone → Locality (from /geo APIs)

Important

- On success, JWT cookie is set automatically
- Redirect user to **Home / Feed**

🔹 2. Login

POST /auth/login

```
{
  "email": "john@gmail.com",
  "password": "password123"
}
```

✅ Cookie set
✅ User is authenticated

🔹 **3. Get Logged-in User**

GET /users/me

Use this to

- Check if user is logged in
- Know cityId, zoneId, localityId
- Know user name for UI

Call this on app load

🌍 **GEO DATA (FOR DROPDOWNS)**

🔹 **Get Cities**

GET /geo/cities

🔹 **Get Zones of City**

GET /geo/zones?cityId=<CITY_ID>

🔹 **Get Localities of Zone**

GET /geo/localities?zoneId=<ZONE_ID>

Frontend

- Cascading dropdowns
- Required during signup

🏷️ **ISSUE CATEGORIES (VERY IMPORTANT)**

🔹 **Get Categories**

GET /issues/categories

Response

```
[
 {
  "id": "uuid",
  "name": "Garbage",
  "department": "BBMP"
```

```
  }
]
```

Frontend

- Use this to show **category chips / dropdown**
- **Send categoryId to backend**, not category name

📝 CREATE ISSUE (MAIN FLOW)

🔹 Create Issue (NO MEDIA)

POST /issues

```
{
  "title": "Garbage not collected",
  "description": "Overflowing bins for 3 days",
  "categoryId": "<CATEGORY_ID>"
}
```

Response

```
{
  "issueId": "UUID"
}
```

⚠️ IMPORTANT

- Save this issueId
- You NEED it for media upload

📸 MEDIA UPLOAD (CLOUDINARY)

🔹 Upload Media

POST /issues/{issueId}/media

Body

- form-data
- key: file
- value: image/video

Frontend Flow

- Same “Submit Issue” button

- First call POST /issues
- Then immediately call /media
- User sees it as **one action**

📰 FEEDS

🔹 Citizen Local Feed

GET /issues/feed

Shows:

- Issues from user's locality

🔹 Universal / Explore Feed

GET /issues/explore

Shows:

- All issues in city

Use this for

- Explore page
- Trending issues

📦 ISSUE CARD UI (VERY IMPORTANT)

Each issue card should show:

- Title
- Category (tag)
- Status (badge)
- Locality name
- Media preview (carousel if multiple)
- 👍 Upvote button
- 💬 Comment button

👍 UPVOTES

🔹 Toggle Upvote

POST /issues/{issueId}/upvote

Frontend

- Single button
- Clicking again removes upvote
- Refresh count from feed

💬 COMMENTS

🔹 Add Comment

POST /issues/{issueId}/comments

```
{
  "content": "This is serious"
}
```

🔹 Get Comments

GET /issues/{issueId}/comments

Frontend

- Modal / bottom sheet
- Show list of comments

👩‍💼 ADMIN FLOW (ADMIN USERS ONLY)

🔹 Admin Feed

GET /admin/issues

Shows:

- Issues assigned to admin's department & locality
- Sorted by SLA urgency

Admin Card Should Show

- Title
- Category
- Status
- SLA deadline (countdown UI 🔥)

🔹 **Admin Update Status**

PATCH /admin/issues/{issueId}/status

```
{
  "status": "IN_PROGRESS",
  "remarks": "Team assigned"
}
```

OR

```
{
  "status": "RESOLVED_PENDING_USER",
  "remarks": "Issue fixed"
}
```

✅ CITIZEN VERIFICATION (FINAL STEP)

🔹 **Verify Issue**

POST /issues/{issueId}/verify

```
{
  "feedback": "Resolved properly"
}
```

⚠️ Only available when status is RESOLVED_PENDING_USER

Frontend

- Show “Verify Issue” button
- Only to issue creator
- After success → status becomes CLOSED

🔁 COMPLETE DATA FLOW (VERY IMPORTANT)

Citizen Flow

Signup/Login

→ View Feed

→ Create Issue

→ Upload Media

→ Upvote / Comment

→ Wait for Admin

→ Verify Issue

Admin Flow

Login

→ View Admin Feed

→ Mark IN_PROGRESS

→ Mark RESOLVED_PENDING_USER

⚠️ IMPORTANT FRONTEND RULES

1. Always use withCredentials: true
2. Never hardcode IDs
3. Always fetch categories from API
4. Issue creation ≠ media upload (two calls)
5. Follow status order strictly
6. Show buttons conditionally based on status

🎤 ONE LINE TO REMEMBER

"Backend enforces workflow. Frontend should guide the user through it."